Metasploit Integration — How to Use

Cerberus pentest framework · driving Metasploit via msfrpcd

operator guide

1. Overview

The framework runs Metasploit **auxiliary modules** (scanners, checks, discovery) against in-scope targets through the `msfrpcd` RPC daemon. Communication is plain XML-RPC over HTTP(S) using only the Python standard library — no new dependencies, no Metasploit database required (execution is console-driven).

2. Prerequisites

- Metasploit Framework installed on the host that runs the backend (or a reachable host running `msfrpcd`).
- The backend service account can reach `MSF_RPC_HOST:MSF_RPC_PORT`.

3. Start the Metasploit RPC daemon

```
msfrpcd -P <your-password> -S -a 127.0.0.1
```

- `-P` — the RPC password (choose a strong one)
- `-S` — disable SSL if you run plain HTTP locally; omit it and set `MSF_RPC_SSL=true` for TLS (the daemon ships a self-signed cert, which the adapter accepts — the channel is authenticated by the password)
- `-a` — bind address

4. Configure the backend

Add to `backend/.env`:

```
MSF_ENABLED=true
MSF_RPC_HOST=127.0.0.1
MSF_RPC_PORT=55553
MSF_RPC_PASSWORD=<your-password>
MSF_RPC_SSL=false
MSF_RPC_TIMEOUT=10
```

> **Security note:** the integration is **opt-in**. With `MSF_ENABLED=false` (the default) the `metasploit_scanner` module simply reports an INFO finding and the engagement continues normally.

5. Run it

The module is a normal framework module — it goes through the `ExecutionEngine`, so scope and Rules of Engagement are enforced before any Metasploit command runs:

```
from app.ai_engine.pentest import ExecutionEngine, PentestContext, ScopeConfig

engine = ExecutionEngine()
context = PentestContext(
    engagement_id="ENG-001",
    target="http://10.0.0.5/",          # must be inside the engagement scope
    scope=ScopeConfig(
        allowed_targets=["10.0.0.5"],   # SafetyGuard refuses anything else
        allowed_ports=[1, 65535],
```

```
    ),
    config={
        "msf_module": "auxiliary/scanner/portscan/tcp",   # optional (default)
        "msf_threads": 10,                                 # optional
        # "msf_options": {"INTERFACE": "eth0"},            # extra module opts
    },
)
findings = await engine.execute("metasploit_scanner", context)
```

6. Module configuration options

Option	Value	Meaning
`RHOSTS`	**automatic**	Set by the module itself from the engagement target — never from user config. The target has already passed the SafetyGuard scope check, so the host Metasploit scans is guaranteed to be in scope.
`msf_module`	`auxiliary/scanner/portscan/tcp`	Any `auxiliary/*` module path. `exploit/*` is refused unless the engagement's RoE ceiling is FULL impact.
`msf_threads`	`10`	Passed to the module as `THREADS` (scan parallelism).
`msf_options`	`{}`	Extra `set KEY VALUE` options forwarded to the module (e.g. `INTERFACE`, `VERSION`).
`msf_timeout`	framework module timeout	Max seconds to wait for console output before the run is considered finished.

7. What you get back

A normal framework `Finding` (severity **INFO**, MITRE **T1046**/T1087) containing:

- Per-port **evidence entries** parsed from console lines like `[*] 10.0.0.5:443 - TCP OPEN` (host, port, protocol, state — CLOSED/FILTERED lines are observations only, not reported as open).
- The **console output tail** (last 4000 chars) as log evidence.
- Metadata: module path, host, open-port list, effective options.

If the tool is disabled or the daemon is unreachable you get an INFO finding explaining why — the engagement itself never fails.

8. Verification

```
# unit tests (15 tool-integration tests, incl. a real fake msfrpcd)
backend\.venv\Scripts\python -m pytest tests/unit/test_pentest_tools.py -q

# full suite
backend\.venv\Scripts\python -m pytest tests/unit -q

# live smoke (expects the INFO "disabled" finding unless you enabled MSF)
backend\.venv\Scripts\python scripts\live_tests\pentest_framework_smoke.py
```

9. Troubleshooting

Symptom	Cause / fix
INFO finding "Metasploit integration disabled"	`MSF_ENABLED=false`. Set it true and restart the backend.
INFO finding "Metasploit unavailable"	`msfrpcd` not running, wrong host/port, or firewall. Start it (step 3) and check `MSF_RPC_HOST` / `MSF_RPC_PORT`.
ERROR finding "login failed"	`MSF_RPC_PASSWORD` doesn't match the `-P` value used at daemon start.
ERROR finding "Exploit module refused"	Correct behavior — exploit modules need FULL impact in the RoE. Run an auxiliary module instead.

Empty port list	The module produced no `OPEN` lines for the target — check the console-output evidence in the finding for the module's own error text (wrong module name, missing options, etc.).

10. Adding the next tool

The layer is tool-agnostic by design. To add nmap / sqlmap / nuclei:

1. Create `pentest/tools/<tool>_adapter.py` implementing `ToolAdapter` (mechanical execution only — no safety logic).
2. Create `pentest/modules/tools/<tool>_module.py` holding the safety gates and output parsing.
3. That's it — both registries auto-discover the new files.

Cerberus backend · pentest framework · Metasploit integration (commit 8fd3935).